

Generative Classifiers

Supplementary reading for Machine Learning I: multinomial logistic regression, LDA, GDA,
Naive Bayes

Alexander Quispe

ENEI – INEI · PEU-CD 2026

September 2026

How to Read This

- Machine Learning I stops at logistic regression, trees, ensembles and PCA. Everything here is *discriminative* $\rightarrow$ *generative*: model $p(\mathbf{x} \mid y)$ and $p(y)$, then classify with Bayes' rule.
- Part 1 extends logistic regression to K classes (softmax) and introduces LDA from the Gaussian side.
- Part 2 develops Gaussian discriminant analysis and the discriminative-vs-generative comparison.
- Part 3 is Naive Bayes for text: bag of words, the conditional-independence assumption, and closed-form estimation.
- Not examined in ML I. Assumed background for the text-as-data material in ML II, Lecture 5.

Outline

Multinomial Logistic Regression & LDA

Generative Classifiers (GDA)

Naive Bayes

Multinomial Logistic Regression & LDA

Multinomial Logistic Regression: Setup

We want to model

$$p_k(x) = \Pr(Y = k \mid X = x), \quad k = 1, \dots, K, \quad \sum_{k=1}^K p_k(x) = 1.$$

Idea: Extend binary logistic regression ($K = 2$) to multiple classes. We must ensure:

- All probabilities $p_k(x)$ are nonnegative.
- They sum to 1 across all classes.

Baseline coding: Choose one class as reference (say class K). We model the log-odds of each class k relative to the baseline:

$$\log \frac{p_k(x)}{p_K(x)} = \beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p, \quad k = 1, \dots, K-1.$$

Deriving the Probability Formula

From the log-odds definition:

$$\frac{p_k(x)}{p_K(x)} = \exp(\beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p).$$

Thus,

$$p_k(x) = p_K(x) \cdot \exp(\beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p).$$

Summing over all classes:

$$1 = \sum_{j=1}^K p_j(x) = p_K(x) \left(1 + \sum_{l=1}^{K-1} \exp(\beta_{l0} + \beta_{l1}x_1 + \cdots + \beta_{lp}x_p) \right).$$

Therefore,

$$p_K(x) = \frac{1}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \cdots + \beta_{lp}x_p}}.$$

And for $k = 1, \dots, K-1$:

$$p_k(x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \cdots + \beta_{lp}x_p}}.$$

Interpretation of Coefficients

Log-odds interpretation:

$$\log \frac{p_k(x)}{p_K(x)} = \beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p.$$

- β_{kj} is the change in the *log-odds* of being in class k vs. baseline K for a one-unit increase in x_j .
- Exponentiating:

$$\frac{p_k(x)}{p_K(x)} \text{ changes by a factor of } e^{\beta_{kj}}$$

for a one-unit increase in x_j .

Example: If baseline is *epileptic seizure*, then:

- $\beta_{\text{stroke},0} = \text{log-odds of stroke vs. seizure when } x = 0.$
- $\beta_{\text{stroke},j} = \text{effect of } x_j \text{ on log-odds of stroke vs. seizure.}$

Softmax Formulation

An alternative (symmetric) parameterization avoids picking a baseline. This is widely used in ML (e.g., neural networks).

Softmax probabilities:

$$p_k(x) = \frac{\exp(\beta_{k0} + \beta_{k1}x_1 + \cdots + \beta_{kp}x_p)}{\sum_{l=1}^K \exp(\beta_{l0} + \beta_{l1}x_1 + \cdots + \beta_{lp}x_p)}, \quad k = 1, \dots, K.$$

Log-odds ratio between class k and k' :

$$\log \frac{p_k(x)}{p_{k'}(x)} = (\beta_{k0} - \beta_{k'0}) + (\beta_{k1} - \beta_{k'1})x_1 + \cdots + (\beta_{kp} - \beta_{k'p})x_p.$$

- This shows that differences in coefficients determine relative odds between any two classes.
- Softmax ensures $\sum_k p_k(x) = 1$ and $p_k(x) \geq 0$.

Generative Models for Classification

- Logistic regression: models $\Pr(Y | X)$ directly (discriminative).
- Generative approach: model $\Pr(X | Y)$ and $\Pr(Y)$, then use Bayes' theorem:

$$\Pr(Y = k | X = x) = \frac{\pi_k f_k(x)}{\sum_{\ell=1}^K \pi_{\ell} f_{\ell}(x)}.$$

- Here:
 - $\pi_k = \Pr(Y = k)$ = prior probability of class k .
 - $f_k(x) = \Pr(X = x | Y = k)$ = class-conditional density.
- Classify to the k that maximizes $\Pr(Y = k | X = x)$ (Bayes classifier).

Linear Discriminant Analysis: Assumptions

- Assume predictor distribution within each class is multivariate normal:

$$X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma), \quad k = 1, \dots, K.$$

- Common covariance matrix Σ across all classes.
- Then,

$$f_k(x) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma^{-1} (x - \mu_k)\right).$$

- Plugging into Bayes' rule yields a classifier with **linear decision boundaries**.

LDA Discriminant Functions

- Posterior log-probability (up to constant):

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k.$$

- LDA rule:

$$\hat{Y} = \arg \max_k \delta_k(x).$$

- Properties:

- Boundaries between classes are hyperplanes (linear in x).
- Incorporates both class means μ_k and priors π_k .
- Effective when p is moderate and normality assumption holds.

LDA Assumptions and Gaussian Density

We want to classify an observation $X = x$ into one of K classes.

Key Assumption

The class-conditional density is Gaussian:

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right).$$

- μ_k : class mean, σ_k^2 : class variance.
- For simplicity, assume equal variance across classes:

$$\sigma_1^2 = \dots = \sigma_K^2 = \sigma^2.$$

This assumption leads to the **Linear Discriminant Analysis (LDA)** formulation.

Posterior Probabilities via Bayes' Rule

From Bayes' theorem, the posterior probability is

$$p_k(x) = \Pr(Y = k \mid X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)},$$

where

- $\pi_k = \Pr(Y = k)$ is the class prior,
- $f_k(x)$ is the class-conditional density.

Plugging in the Gaussian assumption:

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_k)^2\right)}{\sum_{l=1}^K \pi_l \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_l)^2\right)}.$$

Linear Discriminant Functions

We assign x to the class with largest $p_k(x)$.

Define the discriminant function by taking log of the numerator:

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k).$$

- Classification rule: assign x to class with

$$\delta_k(x) = \max_j \delta_j(x).$$

- The decision boundary between class 1 and class 2 occurs when

$$\delta_1(x) = \delta_2(x).$$

$$x = \frac{\mu_1 + \mu_2}{2}.$$

Estimating Parameters for LDA

In practice, the parameters are unknown.

We estimate them from training data:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i: y_i = k} x_i, \quad \hat{\sigma}^2 = \frac{1}{n - K} \sum_{k=1}^K \sum_{i: y_i = k} (x_i - \hat{\mu}_k)^2,$$
$$\hat{\pi}_k = \frac{n_k}{n}.$$

LDA Classifier

Plugging in the estimates, the discriminant becomes

$$\hat{\delta}_k(x) = x \cdot \frac{\hat{\mu}_k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k).$$

Assign x to the class with the largest $\hat{\delta}_k(x)$.

LDA on Three Gaussian Classes (ESL, Figure 4.4)

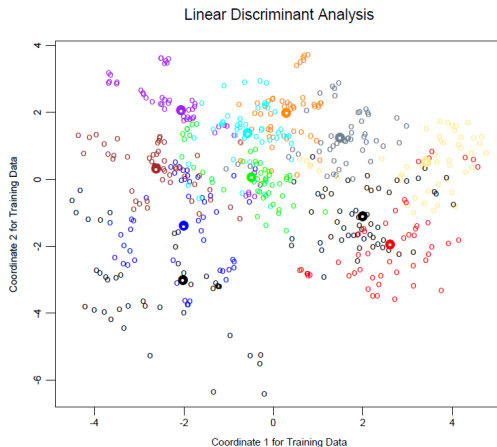


FIGURE 4.4. A two-dimensional plot of the vowel training data. There are eleven classes with $X \in \mathbb{R}^{10}$, and this is the best view in terms of a LDA model (Section 4.3.3). The heavy circles are the projected mean vectors for each class. The class overlap is considerable.

Linear Boundaries From Quadratic Features (ESL, Figure 4.5)

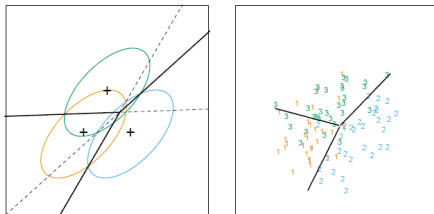


FIGURE 4.5. The left panel shows three Gaussian distributions, with the same covariance and different means. Included are the contours of constant density enclosing 95% of the probability in each case. The Bayes decision boundaries between each pair of classes are shown (broken straight lines), and the Bayes decision boundaries separating all three classes are the thicker solid lines (a subset of the former). On the right we see a sample of 30 drawn from each Gaussian distribution, and the fitted LDA decision boundaries.

Figure 2: Elements of Statistical Learning, 2017

Generative Classifiers (GDA)

Generative Classifiers

In this lecture, we are going to look at generative classifiers and their applications.

We will continue using the Iris dataset for classification. The task is to classify subspecies of Iris flowers based on their measurements:

```
import numpy as np; np.set_printoptions(precision=2)
import pandas as pd; pd.options.display.float_format = "{:,.2f}".format
import warnings; warnings.filterwarnings('ignore')
from sklearn import datasets

iris = datasets.load_iris(as_frame=True)
iris_X, iris_y = iris.data, iris.target
pd.concat([iris_X, iris_y], axis=1).sample(5, random_state=0)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
114	5.80	2.80	5.10	2.40	2
62	6.00	2.20	4.00	1.00	1
33	5.50	4.20	1.40	0.20	0
107	7.30	2.90	6.30	1.80	2

Generative Classifiers

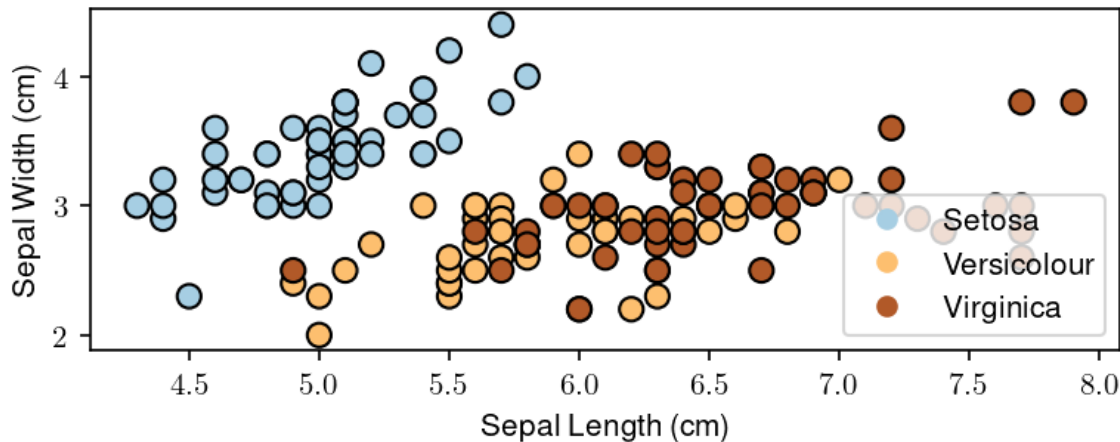
If we only consider the first two feature columns, we can visualize the dataset in 2D.

```
import matplotlib.pyplot as plt
plt.rcParams.update({ "figure.figsize": [6, 2], "figure.dpi": 150, "

# create 2d version of dataset
X = iris_X.to_numpy()[ : , :2]
x_min, x_max = X[ : , 0].min() - .5, X[ : , 0].max() + .5
y_min, y_max = X[ : , 1].min() - .5, X[ : , 1].max() + .5

# Plot also the training points
p1 = plt.scatter(X[ : , 0], X[ : , 1], c=iris_y, edgecolor='k', s=60, cm
plt.xlabel('Sepal Length (cm)'); plt.ylabel('Sepal Width (cm)')
plt.legend(handles=p1.legend_elements()[0], labels=['Setosa', 'Versi
```

Generative Classifiers



Review: Supervised Learning Models

A supervised learning model can be seen as directly predicting a one-hot embedding of the target class:

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\text{input}} \rightarrow \text{model } f_{\theta}(x) \rightarrow \underbrace{\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}}_{\text{output}}$$

In this example, the $f_{\theta}(x)$ predicts the second class for input x .

Review: Probabilistic Models

A probabilistic model outputs a vector of class probabilities

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\text{input}} \rightarrow \text{model } P_{\theta}(y|x) \rightarrow \underbrace{\begin{bmatrix} 0.1 \\ 0.7 \\ 0.2 \end{bmatrix}}_{\text{output}}$$

Here, the $P_{\theta}(y|x)$ still predicts the second class for input x .

Example: A logistic (softmax) model

This model outputs a probability distribution

$$P_{\theta}(y|x) = \begin{bmatrix} P_{\theta}(y=0|x) \\ P_{\theta}(y=1|x) \end{bmatrix} = \begin{bmatrix} 1 - \sigma(\theta^{\top}x) \\ \sigma(\theta^{\top}x) \end{bmatrix}$$

where $\theta^{\top}x$ is a linear model and

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

is the *sigmoid* or *logistic* function.

Example: A logistic (softmax) model

Let's train logistic/softmax regression on this dataset.

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(C=1e5)
```

```
# Create an instance of Logistic Regression Classifier and fit the data
X = iris_X.to_numpy()[ : , :2]
Y = iris_y.copy()
logreg.fit(X, Y);
```

Example: A logistic (softmax) model

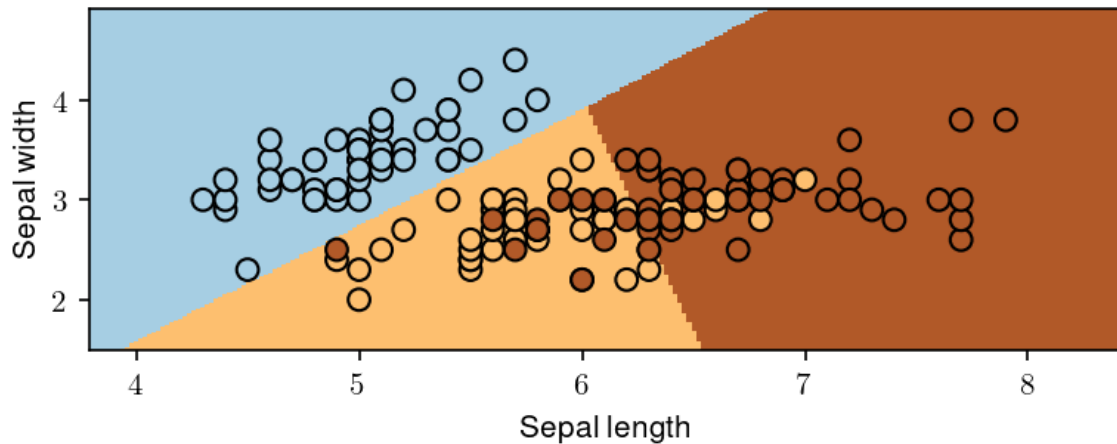
We visualize the regions predicted to be associated with the blue, brown, and yellow classes and the lines between them are the decision boundaries.

```
xx, yy = np.meshgrid(np.arange(x_min, x_max, .02), np.arange(y_min,
Z_lr = logreg.predict(np.c_[xx.ravel(), yy.ravel()]))

# Put the result into a color plot
Z_lr = Z_lr.reshape(xx.shape)
plt.pcolormesh(xx, yy, Z_lr, cmap=plt.cm.Paired)

# Plot also the training points
plt.scatter(X[:, 0], X[:, 1], c=Y, edgecolors='k', cmap=plt.cm.Paired)
plt.xlabel('Sepal length'); plt.ylabel('Sepal width');
```

Example: A logistic (softmax) model



Discriminative vs. Generative Models

Logistic regression is an example of a *discriminative* classifier.

$$P_{\theta}(y|x) = \begin{bmatrix} P_{\theta}(y=0|x) \\ P_{\theta}(y=1|x) \end{bmatrix}$$

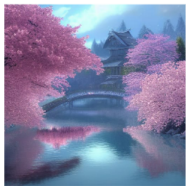
- It directly transforms x into a score for each class y , e.g., via the formula $\sigma(\theta^{\top} x)$
- It can be interpreted as defining a **conditional** probability $P_{\theta}(y|x)$

Generative classifiers instead define distributions $P_{\theta}(x|y)$ and $P_{\theta}(y)$ over x, y .

Relation to text-to-image (TTI) models

- TTI models can also be written as $P(x|y)$, where x is an image (millions of dimensions) and y is a text description of it (many possibilities).
- **This lecture:** x is a flower representation (4D), y is a class (3 choices)

Stable Diffusion



DALLE 2



Midjourney



Stable Diffusion



DALLE 2



Midjourney

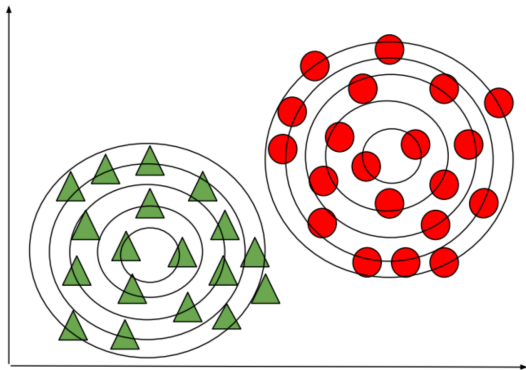


Cherry Blossom near a lake, snowing

Pyramid shaped mountain above a still lake, covered with snow

Generative Classifiers: Intuition

Imagine we have a dataset (e.g., flowers). We can fit a Gaussian to each class:



At test time, we predict the class of the Gaussian that is most likely to have generated the data.

Generative Classifiers: Intuition

Given a new flower x' , we would compare the probabilities of three models:

$$P_{\theta}(x'|y=0)P_{\theta}(y=0)$$

$$P_{\theta}(x'|y=1)P_{\theta}(y=1)$$

$$P_{\theta}(x'|y=2)P_{\theta}(y=2)$$

We output the class that's more likely to have generated x' .

Generative Classifiers: Details

Another approach to classification is to use *generative* models.

- A generative approach first builds a model of x for each class:

$$P_{\theta}(x|y = k) \text{ for each class } k.$$

$P_{\theta}(x|y = k)$ *scores* each x according to how well it matches class k .

- A class probability $P_{\theta}(y = k)$ encoding our prior beliefs. These are often just the % of each class in the data.

Generative Classifiers: Details

In the context of flower classification (e.g., Setosa vs. non-Setosa), we would fit two models on a corpus of emails x with spam/non-spam labels y :

$$P_{\theta}(x|y=0)$$

and

$$P_{\theta}(x|y=1)$$

- $P_{\theta}(x|y=1)$ **scores** each x based on how much it looks like Iris Setosa.
- $P_{\theta}(x|y=0)$ **scores** each x based on whether it looks like non-Setosa.

We also choose a prior $P(y)$.

Given a new x' , we would compare the probabilities of both models:

$$P_{\theta}(x'|y=0)P_{\theta}(y=0)$$

vs.

$$P_{\theta}(x'|y=1)P_{\theta}(y=1)$$

We output the class that's more likely to have generated x' .

Gaussian Mixture Models

Next, we will define another generative model: Gaussian mixtures.

A Generative Classifier for Iris Flowers

To define a generative classifier for Iris flowers, we need to define three probabilities:

$$P_{\theta}(x|y=0)$$

$$P_{\theta}(x|y=1)$$

$$P_{\theta}(x|y=2)$$

We also define priors $P_{\theta}(y=0)$, $P_{\theta}(y=1)$, $P_{\theta}(y=2)$.

How do we choose $P_{\theta}(x|y=k)$ and $P_{\theta}(y)$?

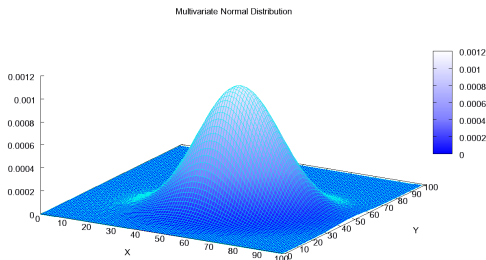
Review: Normal (Gaussian) Distribution

A multivariate normal distribution with parameters $\theta = (\mu, \Sigma)$ is a probability over a d -dimensional $x \in \mathbb{R}^d$

$$P_{\theta}(x; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp \left(-\frac{1}{2} (x - \mu)^{\top} \Sigma^{-1} (x - \mu) \right)$$

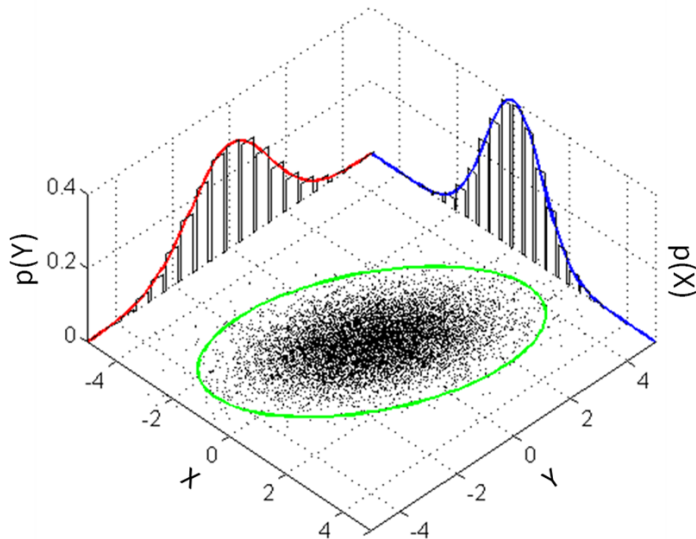
In one dimension, this reduces to $\frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(x-\mu)^2}{2\sigma^2} \right)$.

This is what the density of a Normal distribution looks like in 2D:



Review: Normal (Gaussian) Distribution

This is how we can visualize it in a 2D plane:



Gaussian Mixture Model: Definition

A **Gaussian mixture** model (GMM) $P_\theta(x, y)$ is defined for **real-valued data** $x \in \mathbb{R}^d$.

The θ contains prior parameters $\vec{\phi} = (\phi_1, \dots, \phi_K)$ and K sets of per-class Gaussian parameters μ_k, Σ_k .

The probability of the data x for each class is a multivariate Gaussian

$$P_\theta(x|y = k) = \mathcal{N}(x; \mu_k, \Sigma_k).$$

The probability over y is Categorical: $P_\theta(y = k) = \phi_k$.

Gaussian Discriminant Analysis

Next, we will use GMMs as the basis for a new generative classification algorithm, Gaussian Discriminant Analysis (GDA).

The Parameters of a GMM

We need to learn the parameters of two sets of distributions:

- The distribution over classes is Categorical, denoted $\text{Categorical}(\phi_1, \phi_2, \dots, \phi_K)$. Thus, $P_{\theta}(y = k) = \phi_k$.
- The conditional probability $P(x | y = k)$ of the data under class k is a multivariate Gaussian $\mathcal{N}(x; \mu_k, \Sigma_k)$ with mean and covariance μ_k, Σ_k .

Learning the Parameters μ_k, Σ_k

- Consider the dataset of all the points x for which $y = k$.
- Goal: learn the mean and variance $\theta_k = (\mu_k, \Sigma_k)$ of a Normal that generates x
- What is the maximum likelihood μ_k, Σ_k in this case? The formulation is:

$$\arg \max_{\theta_k} P_{\theta_k}(x|y = k)$$

Gaussian Discriminant Analysis

- For Gaussians, these are the empirical means and covariances of each class.

$$\mu_k = \frac{\sum_{i: y^{(i)}=k} x^{(i)}}{n_k}$$
$$\Sigma_k = \frac{\sum_{i: y^{(i)}=k} (x^{(i)} - \mu_k)(x^{(i)} - \mu_k)^\top}{n_k}$$

Learning the Parameters ϕ

Next, consider learning $\vec{\phi} = (\phi_1, \phi_2, \dots, \phi_K)$.

- We have n datapoints. Each point has a label $k \in \{1, 2, \dots, K\}$.
- Our model is a categorical and assigns a probability ϕ_k to each outcome $k \in \{1, 2, \dots, K\}$.
- We want to infer ϕ_k assuming our dataset is sampled from the model.

What are the maximum likelihood ϕ_k that are most likely to have generated our data?

Gaussian Discriminant Analysis

Intuitively, the maximum likelihood class probabilities ϕ should just be the class proportions that we see in the data.

$$\phi_k = \frac{n_k}{n}$$

for each k , where $n_k = |\{i: y^{(i)} = k\}|$ is the number of training targets with class k . Thus, the optimal ϕ_k is just the proportion of data points with class k in the training set!

Querying the Model

How do we ask the model for predictions? As discussed earlier, we can apply Bayes' rule:

$$\arg \max_y P_\theta(y|x) = \arg \max_y P_\theta(x|y)P(y).$$

Thus, we can estimate the probability of x and under each $P_\theta(x|y = k)P(y = k)$ and choose the class that explains the data best.

Example: Iris Flower Classification

Let's see how this approach can be used in practice on the Iris dataset.

- We will learn the maximum likelihood GDA parameters
- We will compare the outputs to the true predictions.

Let's first start by computing the true parameters on our dataset.

```
# we can implement these formulas over the Iris dataset
d = 2 # number of features in our toy dataset
K = 3 # number of classes
n = X.shape[0] # size of the dataset

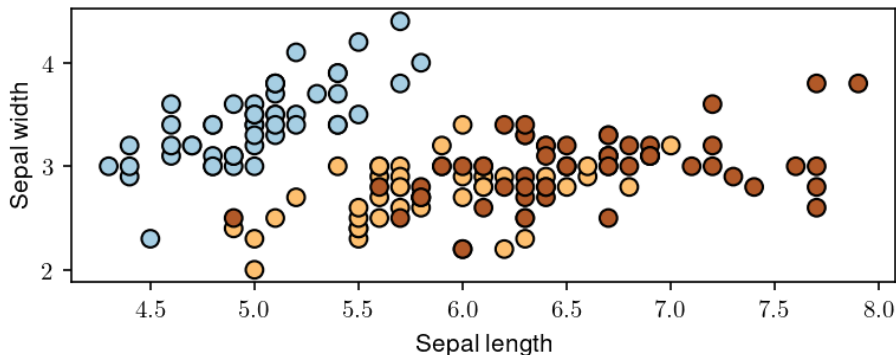
# these are the shapes of the parameters
mus = np.zeros([K,d])
Sigmas = np.zeros([K,d,d])
phis = np.zeros([K])

# we now compute the parameters
for k in range(3):
    X_k = X[iris_y == k]
```

Example: Iris Flower Classification

```
Sigmas[k] = np.cov(X_k.T)  
phis[k] = X_k.shape[0] / float(n)
```

```
plt.scatter(X[:, 0], X[:, 1], c=Y, edgecolors='k', cmap=plt.cm.Paired)  
plt.xlabel('Sepal length'); plt.ylabel('Sepal width');
```



Example: Iris Flower Classification

We can compute predictions using Bayes' rule.

```
def gda_predictions(x, mus, Sigmas, phis):  
    """ This returns class assignments and  $p(y|x)$  under the GDA model
```

```
    We compute  $\arg\max_y p(y|x)$  as  $\arg\max_y p(x|y)p(y)$   
    """
```

```
    # adjust shapes
```

```
    n, d = x.shape
```

```
    x = np.reshape(x, (1, n, d, 1))
```

```
    mus = np.reshape(mus, (K, 1, d, 1))
```

```
    Sigmas = np.reshape(Sigmas, (K, 1, d, d))
```

```
    # compute probabilities
```

```
    py = np.tile(phis.reshape((K,1)), (1,n)).reshape([K,n,1,1])
```

```
    pxy = (
```

Example: Iris Flower Classification

```
np.sqrt(np.abs((2*np.pi)**d*np.linalg.det(Sigmas))).reshape((K,
    * -.5*np.exp(
        np.matmul(np.matmul((x-mus).transpose([0,1,3,2]), np.lin
    )
)
pyx = pxy * py
return pyx.argmax(axis=0).flatten(), pyx.reshape([K,n])
```

```
idx, pyx = gda_predictions(X, mus, Sigmas, phis)
print(idx)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 1 0 0 0 0 0 0 0 0 2 2 2 1 2 1 2 1 2 1 1 1 1 1 1 2 1 1 1 1 1 2 1  
2 2 2 2 1 1 1 1 1 1 1 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 2 2 2 1 2 2 2 2  
2 2 1 1 2 2 2 2 1 2 1 2 1 2 2 1 1 2 2 2 2 2 1 1 2 2 2 1 2 2 2 1 2 2 2 2 2  
2 1]
```

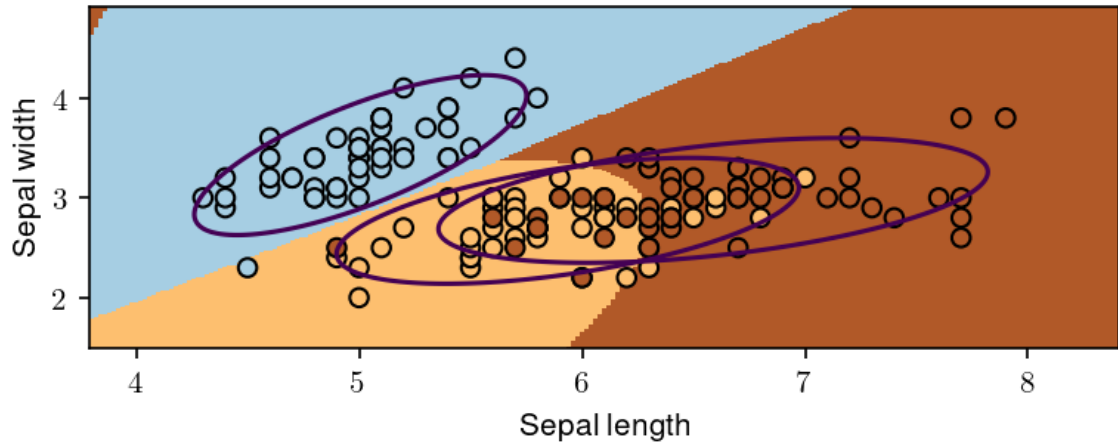
Example: Iris Flower Classification

We visualize the decision boundaries like we did earlier.

```
Z, pyx = gda_predictions(np.c_[xx.ravel(), yy.ravel()], mus, Sigmas,  
logpy = np.log(-1./3*pyx)
```

```
Z = Z.reshape(xx.shape)  
contours = np.zeros([K, xx.shape[0], xx.shape[1]])  
for k in range(K): contours[k] = logpy[k].reshape(xx.shape)  
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)  
for k in range(K): plt.contour(xx, yy, contours[k], levels=np.logspa  
  
plt.scatter(X[:, 0], X[:, 1], c=iris_y, edgecolors='k', cmap=plt.cm.  
plt.xlabel('Sepal length'); plt.ylabel('Sepal width');
```

Example: Iris Flower Classification



Discriminative vs. Generative Classifiers

We conclude our lecture on generative classifiers by revisiting the question of how they compare to discriminative classifiers.

Linear Discriminant Analysis

When the covariances Σ_k in GDA are equal, we have an algorithm called Linear Discriminant Analysis or LDA.

The probability of the data x for each class is a multivariate Gaussian with the same covariance Σ .

$$P_{\theta}(x|y = k) = \mathcal{N}(x; \mu_k, \Sigma).$$

The probability over y is Categorical: $P_{\theta}(y = k) = \phi_k$.

Let's try this algorithm on the Iris flower dataset.

We compute the model parameters similarly to how we did for GDA.

Linear Discriminant Analysis

```
# we can implement these formulas over the Iris dataset
d = 2 # number of features in our toy dataset
K = 3 # number of classes
n = X.shape[0] # size of the dataset
```

```
# these are the shapes of the parameters
mus = np.zeros([K,d])
Sigmas = np.zeros([K,d,d])
phis = np.zeros([K])
```

```
# we now compute the parameters
for k in range(3):
    X_k = X[iris_y == k]
    mus[k] = np.mean(X_k, axis=0)
    Sigmas[k] = np.cov(X.T) # this is now X.T instead of X_k.T
    phis[k] = X_k.shape[0] / float(n)
```


Linear Discriminant Analysis

We can compute predictions using Bayes' rule.

```
def gda_predictions(x, mus, Sigmas, phis):  
    """This returns class assignments and  $p(y|x)$  under the GDA model
```

```
  
    We compute  $\arg\max_y p(y|x)$  as  $\arg\max_y p(x|y)p(y)$   
    """
```

```
    # adjust shapes
```

```
    n, d = x.shape
```

```
    x = np.reshape(x, (1, n, d, 1))
```

```
    mus = np.reshape(mus, (K, 1, d, 1))
```

```
    Sigmas = np.reshape(Sigmas, (K, 1, d, d))
```

```
    # compute probabilities
```

```
    py = np.tile(phis.reshape((K,1)), (1,n)).reshape([K,n,1,1])
```

```
    pxy = (  
        np.sqrt(np.abs((2*np.pi)**d*np.linalg.det(Sigmas)))).reshape(  
            [K,n,1,1])
```

Linear Discriminant Analysis

```
* -.5*np.exp(
    np.matmul(np.matmul((x-mus).transpose([0,1,3,2]), np.linalg.pinv(Sigma))
)
)
pyx = pxy * py
return pyx.argmax(axis=0).flatten(), pyx.reshape([K,n])
```

```
idx, pyx = gda_predictions(X, mus, Sigmas, phis)
print(idx)
```

[illegible]

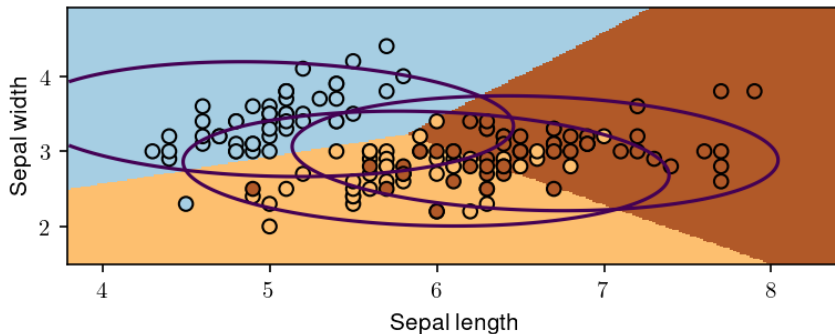
Linear Discriminant Analysis

We visualize predictions like we did earlier.

```
Z, pyx = gda_predictions(np.c_[xx.ravel(), yy.ravel()], mus, Sigmas,  
logpy = np.log(-1./3*pyx)
```

```
Z = Z.reshape(xx.shape)  
contours = np.zeros([K, xx.shape[0], xx.shape[1]])  
for k in range(K):  
    contours[k] = logpy[k].reshape(xx.shape)  
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)  
for k in range(K):  
    plt.contour(xx, yy, contours[k], levels=np.logspace(0, 1, 1))  
  
plt.scatter(X[:, 0], X[:, 1], c=iris_y, edgecolors='k', cmap=plt.cm.  
plt.xlabel('Sepal length'); plt.ylabel('Sepal width');
```

Linear Discriminant Analysis



Linear Discriminant Analysis outputs decision boundaries that are linear, just like Logistic/Softmax Regression.

Softmax or Logistic regression also produce linear boundaries. In fact, both types of algorithms make use of the same model class.

Are the algorithms equivalent?

What Is the LDA Model Class?

We can derive a formula for $P_{\theta}(y|x)$ in a Bernoulli Naive Bayes or LDA model when $K = 2$:

$$P_{\theta}(y|x) = \frac{P_{\theta}(x|y)P_{\theta}(y)}{\sum_{y' \in \mathcal{Y}} P_{\theta}(x|y')P_{\theta}(y')} = \frac{1}{1 + \exp(-\gamma^{\top} x)}$$

for some set of parameters γ (whose expression can be derived from θ).

This is the same form as Logistic Regression! Does it mean that the two sets of algorithms are equivalent?

No! They assume the same model class $\mathcal{M}$, they use a different objective J to select a model in $\mathcal{M}$.

What Is the LDA Model Class?

```
nrow, ncol = 1, 2
```

```
fig, axs = plt.subplots(nrow, ncol, figsize=(6*ncol, 2*nrow))
```

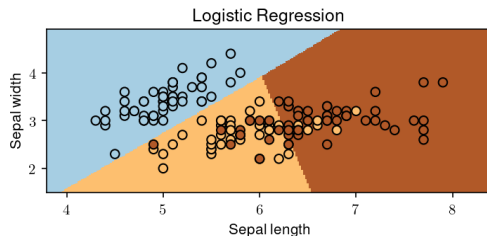
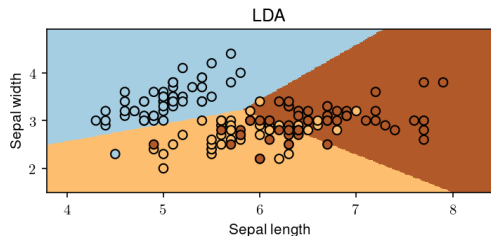
```
axs[0].pcolormesh(xx, yy, Z, cmap=plt.cm.Paired); axs[0].set_title('LDA')
```

```
axs[1].pcolormesh(xx, yy, Z_lr, cmap=plt.cm.Paired); axs[1].set_title('Logistic Regression')
```

```
for ax in axs:
```

```
    ax.scatter(X[:, 0], X[:, 1], c=iris_y, edgecolors='k', cmap=plt.cm.Paired)
```

```
    ax.set_xlabel('Sepal length'); ax.set_ylabel('Sepal width')
```



LDA vs. Logistic Regression

What are the differences between LDA/NB and logistic regression?

- Bernoulli Naive Bayes or LDA assumes a logistic form for $P(y|x)$. But the converse is not true: logistic regression does not assume a NB or LDA model for $P(x, y)$.
- Generative models make stronger modeling assumptions. If these assumptions hold true, the generative models will perform better.
- But if they don't, logistic regression will be more robust to outliers and model misspecification, and achieve higher accuracy.

Discriminative Approaches

Discriminative algorithms are deservedly very popular.

- Most state-of-the-art algorithms for classification are discriminative (including neural nets, boosting, SVMs, etc.)
- They are often more accurate because they make fewer modeling assumptions.

Other Useful Features of Generative Models

Generative models can also do things that discriminative models can't do.

- **Generation:** we can sample $x \sim p(x|y)$ to generate new data (images, audio).
- **Missing value imputation:** if x_j is missing, we infer it using $p(x|y)$.
- **Outlier detection:** we may detect via $p(x')$ if x' is an outlier.
- **Scalability:** Simple formulas for maximum likelihood parameters.

Naive Bayes

Text Classification

We will now do a quick detour to talk about an important application area of machine learning: text classification. Afterwards, we will see how text classification motivates new classification algorithms.

Review: Classification

Consider a training dataset $\mathcal{D} = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$.

We distinguish between two types of supervised learning problems depending on the targets $y^{(i)}$.

1. **Regression:** The target variable $y \in \mathcal{Y}$ is continuous: $\mathcal{Y} \subseteq \mathbb{R}$.
2. **Classification:** The target variable y is discrete and takes on one of K possible values: $\mathcal{Y} = \{y_1, y_2, \dots, y_K\}$. Each discrete value corresponds to a *class* that we want to predict.

An interesting instance of a classification problem is **classifying text**.

- Many applications: spam filtering, fraud detection, medical record classification.
- Inputs are sequences of words of an arbitrary length.

Feature Representations for Text

Each data point x in this dataset is a sequence of characters of an arbitrary length.

How do we transform these into d -dimensional features $\phi(x)$ that can be used with our machine learning algorithms?

- We may devise hand-crafted features by inspecting the data:
 - Does the message contain the word "*church*"?
 - Does the email of the user originate outside the United States?
 - Is the organization a university?
- We can count the number of occurrences of each word:
 - Does this message contain "*Aardvark*", yes or no?
 - Does this message contain "*Apple*", yes or no?
 - Does this message contain "*Zebra*", yes or no?

Finally, many modern deep learning methods can directly work with sequences of characters of an arbitrary length.

Bag of Words Representations

A widely-used approach to representing text documents is called "*bag of words*".

We start by defining a **vocabulary** V containing all the possible words we are interested in, e.g.:

$$V = \{\text{church, doctor, fervently, purple, slow, ...}\}$$

A bag of words representation of a document x is a function $\phi(x) \rightarrow \{0, 1\}^{|V|}$ that outputs a feature vector

$$\phi(x) = \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 1 \\ \vdots \end{pmatrix} \begin{matrix} \text{church} \\ \text{doctor} \\ \text{fervently} \\ \vdots \\ \text{purple} \\ \vdots \end{matrix}$$

of dimension $|V|$. The j -th component $\phi(x)_j$ equals 1 if x contains the j -th word in V and 0 otherwise.

Classification Dataset: Twenty Newsgroups

To illustrate the text classification problem, we will use a popular dataset called **20-newsgroups**.

- It contains 20,000 documents collected approximately evenly from 20 different online newsgroups.
- Each newsgroup covers a different topic such as medicine, computer graphics, or religion.
- This dataset is often used to benchmark text classification and other types of algorithms.

Classification Dataset: Twenty Newsgroups

```
from IPython.display import Markdown, display
import numpy as np; np.set_printoptions(precision=2)
import pandas as pd; pd.options.display.float_format = "{:,.2f}".format
from sklearn.datasets import fetch_20newsgroups

# for this lecture, we will restrict our attention to just 4 different
categories = ['alt.atheism', 'soc.religion.christian', 'comp.graphics', 'sci.space']

# load the dataset
twenty_train = fetch_20newsgroups(subset='train', categories=categories)

# print some information on it
Markdown(twenty_train.DESCR[:1088])
```

Classification Dataset: Twenty Newsgroups

The 20 newsgroups text dataset comprises around 18000 newsgroups posts on 20 topics split in two subsets: one for training (or development) and the other one for testing (or for performance evaluation). The split between the train and test set is based upon a messages posted before and after a specific date

This module contains two loaders:

- The first one, `sklearn.datasets.fetch_20newsgroups`, returns a list of the raw texts that can be fed to text feature extractors such as `sklearn.feature_extraction.text.CountVectorizer` with custom parameters to extract feature vectors.
- The second one, `sklearn.datasets.fetch_20newsgroups_vectorized`, returns ready-to-use features, i.e., it is not necessary to use a feature extractor.

Classification Dataset: Twenty Newsgroups

Data Set Characteristics:

Classes	20
Samples total	18846
Dimensionality	1
Features	text

```
# The set of targets in this dataset are the newgroup topics:  
twenty_train.target_names
```

```
['alt.atheism', 'comp.graphics', 'sci.med', 'soc.religion.christian']
```

```
# Let's examine one data point  
print(twenty_train.data[3])
```


Classification Dataset: Twenty Newsgroups

From: s0612596@let.rug.nl (M.M. Zwart)
Subject: catholic church poland
Organization: Faculteit der Letteren, Rijksuniversiteit Groningen, NL
Lines: 10

Hello,

I'm writing a paper on the role of the catholic church in Poland after 1989. Can anyone tell me more about this, or fill me in on recent books/articles(in english, german or french). Most important for me is the role of the church concerning the abortion-law, religious education at schools, birth-control and the relation church-state(government). Thanx,

Masja,

"M.M.Zwart"<s0612596@let.rug.nl>

```
# We have about 2k data points in total  
print(len(twenty_train.data))
```

2257

Classification Dataset: Twenty Newsgroups

Let's see an example of Bag-of-Words on **20-newsgroups**.

We start by computing these features using the **sklearn** library.

```
from sklearn.feature_extraction.text import CountVectorizer

# vectorize the training set
count_vect = CountVectorizer(binary=True)
X_train = count_vect.fit_transform(twenty_train.data)
X_train.shape
```

(2257, 35788)

The 'sklearn' count vectorizer gives us mappings between words and indices:

```
print('Word to index '.vocabularly ': ', str(count_vect.vocabulary_))[:
feature_names = count_vect.get_feature_names_out()
print('Index to word: ', feature_names[:10])
```

Classification Dataset: Twenty Newsgroups

```
Word to index `.vocabulary`: {'from': 14887, 'sd345': 29022, 'city': 8696, 'ac': 4017, 'uk': 33256, 'michael': 21661, 'coll
...
Index to word: ['00' '000' '0000' '0000001200' '000005102000' '0001' '000100255pixel'
'00014' '000406' '0007']
```

We can retrieve the index of $\phi(x)$ associated with each 'word' using these mappings:

```
# The CountVectorizer class records the index j associated with each
print('Index for the word "church": ', count_vect.vocabulary_.get(u'
print('Index for the word "computer": ', count_vect.vocabulary_.get(
```

```
# And we can map the indices back to words
print("Word for the index '8609': ", feature_names[8609])
print("Word for the index '10000': ", feature_names[10000])
```

```
Index for the word "church": 8609
Index for the word "computer": 9338
Word for the index '8609': church
Word for the index '10000': counseling
```

Classification Dataset: Twenty Newsgroups

Our featurized dataset is in the matrix `X_train`. We can use the above indices to retrieve the 0-1 value that has been computed for each word:

```
# We can examine if any of these words are present in our previous corpus
print(twenty_train.data[3])
```

```
# let's see if it contains these two words?
```

```
print('---'*20)
print('Value at the index for the word "church": ', X_train[3, count_vect.get('church')])
print('Value at the index for the word "computer": ', X_train[3, count_vect.get('computer')])
print('Value at the index for the word "doctor": ', X_train[3, count_vect.get('doctor')])
print('Value at the index for the word "important": ', X_train[3, count_vect.get('important')])
```

Classification Dataset: Twenty Newsgroups

From: s0612596@let.rug.nl (M.M. Zwart)
Subject: catholic church poland
Organization: Faculteit der Letteren, Rijksuniversiteit Groningen, NL
Lines: 10

Hello,

I'm writing a paper on the role of the catholic church in Poland after 1989. Can anyone tell me more about this, or fill me in on recent books/articles(in english, german or french). Most important for me is the role of the church concerning the abortion-law, religious education at schools, birth-control and the relation church-state(government). Thanx,

Masja,

"M.M.Zwart"<s0612596@let.rug.nl>

Value at the index for the word "church": 1
Value at the index for the word "computer": 0
Value at the index for the word "doctor": 0
Value at the index for the word "important": 1

Practical Considerations

In practice, we may use some additional modifications of this technique:

- Sometimes, the feature $\phi(x)_j$ for the j -th word holds the count of occurrences of word j instead of just the binary occurrence.
- The raw text is usually preprocessed. One common technique is *stemming*, in which we only keep the root of the word.
 - e.g. "slowly", "slowness", both map to "slow".
- Filtering for common *stopwords* such as "the", "a", "and". Similarly, rare words are also typically excluded.

Classification Using BoW Features

Let's now have a look at the performance of classification over bag of words features.

Now that we have a feature representation $\phi(x)$, we can apply the classifier of our choice, such as logistic regression.

```
from sklearn.linear_model import LogisticRegression

# Create an instance of Softmax and fit the data.
logreg = LogisticRegression(C=1e5, multi_class='multinomial', verbose=1)
logreg.fit(X_train, twenty_train.target)
```

Classification Using BoW Features

RUNNING THE L-BFGS-B CODE

* * *

Machine precision = 2.220D-16

N = 143156 M = 10

At X0 0 variables are exactly at the bounds

At iterate 0 f= 3.12887D+03 |proj g|= 2.12500D+02

* * *

Tit = total number of iterations

Tnf = total number of function evaluations

Tnint = total number of segments explored during Cauchy searches

Skip = number of BFGS updates skipped

Nact = number of active bounds at final generalized Cauchy point

Projg = norm of the final projected gradient

F = final function value

* * *

Classification Using BoW Features

And now we can use this model for predicting on new inputs.

```
docs_new = [ 'God is love ', 'OpenGL on the GPU is fast ']
```

```
X_new = count_vect.transform(docs_new)
```

```
predicted = logreg.predict(X_new)
```

```
for doc, category in zip(docs_new, predicted):  
    print('%r => %s' % (doc, twenty_train.target_names[category]))
```

```
'God is love' => soc.religion.christian  
'OpenGL on the GPU is fast' => comp.graphics
```

Generative Classifiers for Text

In this lecture, we are going to look at generative classifiers and their applications to text classification.

Review: Supervised Learning Models A supervised learning model can be seen as directly predicting a one-hot embedding of the target class:

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\text{input}} \rightarrow \text{model } f_{\theta}(x) \rightarrow \underbrace{\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}}_{\text{output}}$$

In this example, the $f_{\theta}(x)$ predicts the second class for input x .

Generative Classifiers for Text

Review: Probabilistic Models A probabilistic model outputs a vector of class probabilities

$$\underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\text{input}} \rightarrow \text{model } P_{\theta}(y|x) \rightarrow \underbrace{\begin{bmatrix} 0.1 \\ 0.7 \\ 0.2 \end{bmatrix}}_{\text{output}}$$

Here, the $P_{\theta}(y|x)$ still predicts the second class for input x .

Generative Classifiers for Text

Review: Logistic Regression For example, a logistic (softmax) model outputs a probability distribution

$$P_{\theta}(y|x) = \begin{bmatrix} P_{\theta}(y=0|x) \\ P_{\theta}(y=1|x) \end{bmatrix} = \begin{bmatrix} 1 - \sigma(\theta^{\top}x) \\ \sigma(\theta^{\top}x) \end{bmatrix}$$

where $\theta^{\top}x$ is a linear model and

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

is the *sigmoid* or *logistic* function.

Discriminative vs. Generative Classifiers

Logistic regression is an example of a *discriminative* machine learning classifier.

$$P_{\theta}(y|x) = \begin{bmatrix} P_{\theta}(y=0|x) \\ P_{\theta}(y=1|x) \end{bmatrix}$$

- It directly transforms x into a score for each class y (e.g., via the formula $\sigma(\theta^{\top}x)$)
- It can be interpreted as defining a *conditional* probability $P_{\theta}(y|x)$

Example: Gaussian Discriminant Analysis

Suppose we want to train a classifier over Iris flowers. In GDA, we define **Gaussian** probabilities:

$$P_{\theta}(x|y = \text{setosa})$$

$$P_{\theta}(x|y = \text{non-setosa})$$

as well as prior probabilities $P_{\theta}(y = \text{setosa})$, $P_{\theta}(y = \text{non-setosa})$.

Each Gaussian $P_{\theta}(x|y = \text{setosa/non-setosa})$ matches the mean and variance of the class. The $P(y)$ measures the frequency of each class y .

Predictions From Generative Classifiers

Given a new x' , we would compare the probabilities of both models:

$$P_{\theta}(x'|y=0)P_{\theta}(y=0) \quad \text{vs.} \quad P_{\theta}(x'|y=1)P_{\theta}(y=1)$$

We output the class that's more likely to have generated x' .

Formally, given a new x' , we return the most likely class to have generated it:

$$\begin{aligned} \arg \max_k P_{\theta}(y=k|x') &= \arg \max_k \frac{P_{\theta}(x'|y=k)P_{\theta}(y=k)}{P_{\theta}(x')} \\ &= \arg \max_k P_{\theta}(x'|y=k)P_{\theta}(y=k), \end{aligned}$$

where we have applied Bayes' rule in the first line.

A Generative Model for Text Classification

In binary text classification, we fit two models on a labeled corpus:

$$P_{\theta}(x|y=0) \qquad \text{and} \qquad P_{\theta}(x|y=1)$$

Each model $P_{\theta}(x|y=k)$ *scores* x based on how much it looks like class k . The documents x are in **bag-of-words** representation.

How do we choose $P_{\theta}(x|y=k)$?

Review: Categorical Distribution

A Categorical distribution with parameters θ is a probability over K discrete outcomes $x \in \{1, 2, \dots, K\}$:

$$P_{\theta}(x = j) = \theta_j.$$

When $K = 2$ this is called the Bernoulli.

First Attempt at a Generative Model

A first solution is to assume that $P(x|y = \text{spam})$ is a categorical distribution that assigns a probability to each possible word $x' = [0, 1, 0, \dots, 0]$:

$$P(x = x' | y = \text{spam}) = P \left(\begin{array}{cc} 0 & \text{church} \\ 1 & \text{doctor} \\ 0 & \text{fervently} \\ \vdots & \vdots \\ 0 & \text{purple} \end{array} \right) = \theta_{x'} = 0.0012$$

The $\theta_{x'}$ is the probability of x' under class spam. We need to specify θ_x for all x .

The Naive Bayes Assumption

The Naive Bayes assumption is a **general technique** that can be used with any d -dimensional x to construct tractable models $P_{\theta}(x|y)$ as follows:

$$P_{\theta} \left(x = \begin{pmatrix} 0 & \text{church} \\ 1 & \text{doctor} \\ 0 & \text{fervently} \\ \vdots & \vdots \\ 0 & \text{purple} \end{pmatrix} \right) = P_{\theta}(x_1 = 0) \cdot P_{\theta}(x_2 = 1) \cdot \dots \cdot P_{\theta}(x_d = 0)$$

Each $P_{\theta}(x_j = 0)$ is a Bernoulli.

The Naive Bayes Assumption

How many parameters do we need to specify this probability?

$$P_{\theta} \left(\begin{array}{cc} 0 & \text{church} \\ 1 & \text{doctor} \\ 0 & \text{fervently} \\ \vdots & \vdots \\ 0 & \text{purple} \end{array} \right) = P_{\theta}(x_1 = 0) \cdot P_{\theta}(x_2 = 1) \cdot \dots \cdot P_{\theta}(x_d = 0)$$

This typically makes the number of parameters linear instead of exponential in d .

Formally, the Naive Bayes assumptions is defined as follows:

- We define the probability that x takes value x' to be a product of independent terms (one for each attribute x_j):

$$P_{\theta}(x = x' | y) = \prod_{j=1}^d P_{\theta}(x_j = x'_j | y)$$

- Each $P_{\theta}(x_j = x'_j | y)$ is typically defined to be a simple distribution like a Bernoulli.

Naive Bayes Assumption for Bag of Words

To deal with high-dimensional x , we choose a simpler model for $P_{\theta}(x|y)$:

1. We define the model $P_{\theta}(x|y = k)$ for documents x as the product of the occurrence probabilities of each of its words x_j :

$$P_{\theta}(x = x' | y = k) = \prod_{j=1}^d P_{\theta}(x_j = x'_j | y = k)$$

2. We define a (Bernoulli) model with one parameter $\psi_{jk} \in [0, 1]$ for the occurrence of each word j in class k :

$$P_{\theta}(x_j = 1 | y = k) = \psi_{jk} \quad P_{\theta}(x_j = 0 | y = k) = 1 - \psi_{jk}$$

ψ_{jk} is the probability that a document of class k contains word j .

Naive Bayes Assumption for Bag of Words

How many parameters does this new model have?

- We have K distributions of the form $P_{\theta}(x|y = k)$.
- The distribution $P_{\theta}(x|y = k) = \prod_{j=1}^d P_{\theta}(x_j | y = k)$ is the product of d such one-parameter distributions.
- We have a distribution $P_{\theta}(x_j = 1 | y = k)$ for each word j and each distribution has one parameter ψ_{jk} .

Thus, we only need Kd parameters instead of $K(2^d - 1)!$

Is Naive Bayes a Good Assumption?

Naive Bayes assumes that words are uncorrelated, but in reality they are.

- If spam email contains "bank", it probably contains "account"

As a result, the probabilities estimated by Naive Bayes can be over- under under-confident.

In practice, however, Naive Bayes is a very useful assumption that gives very good classification accuracy!

Defining Prior Distributions

A full generative model still requires defining the distribution $P_{\theta}(y = k)$.

- This encodes our prior belief about y before we see x .
- It can also be learned from data.

Since we have a small number of classes K , we may use a Categorical distribution with parameters $\vec{\phi} = (\phi_1, \dots, \phi_K)$ and learn $\vec{\phi}$ from data:

$$P_{\theta}(y = k) = \phi_k.$$

Bernoulli Naive Bayes Model

The *Bernoulli Naive Bayes* model $P_{\theta}(x, y)$ is defined for *binary data* $x \in \{0, 1\}^d$ (e.g., bag-of-words documents).

The θ contains prior parameters $\vec{\phi} = (\phi_1, \dots, \phi_K)$ and K sets of per-class parameters $\psi_k = (\psi_{1k}, \dots, \psi_{dk})$.

- The probability of the data x for each class equals

$$P_{\theta}(x|y = k) = \prod_{j=1}^d P(x_j | y = k),$$

where each $P_{\theta}(x_j | y = k)$ is a Bernoulli(ψ_{jk}).

- The probability over y is Categorical: $P_{\theta}(y = k) = \phi_k$.

Bernoulli Naive Bayes Model

Formally, we have:

$$\begin{aligned}P_{\theta}(y) &= \text{Categorical}(\phi_1, \phi_2, \dots, \phi_K) \\P_{\theta}(x_j = 1|y = k) &= \text{Bernoulli}(\psi_{jk}) \\P_{\theta}(x|y = k) &= \prod_{j=1}^d P_{\theta}(x_j|y = k)\end{aligned}$$

The parameters of the model are $\theta = (\phi_1, \dots, \phi_K, \psi_{11}, \dots, \psi_{dK})$. There are exactly $K(d+1)$ parameters.

The Parameters of a Naive Bayes Model

We need to learn the parameters of two sets of distributions:

- The distribution over classes is Categorical, denoted $\text{Categorical}(\phi_1, \phi_2, \dots, \phi_K)$. Thus, $P_\theta(y = k) = \phi_k$.
- The conditional probability $P(x \mid y = k)$ of the data under class k is a product of Bernoullis

$$P_\theta(x \mid y = k) = \prod_{j=1}^d P(x_j \mid y = k),$$

where each $P_\theta(x_j \mid y = k)$ is a Bernoulli(ψ_{jk}).

Learning the Parameters ϕ

Consider learning $\vec{\phi} = (\phi_1, \phi_2, \dots, \phi_K)$.

- We have n datapoints. Each point has a label $k \in \{1, 2, \dots, K\}$.
- Our model is a categorical and assigns a probability ϕ_k to each outcome $k \in \{1, 2, \dots, K\}$.
- We want to infer ϕ_k assuming our dataset is sampled from the model.

What are the maximum likelihood ϕ_k that are most likely to have generated our data?

Intuitively, the class probabilities ϕ should just be the class proportions that we see in the data.

$$\phi_k = \frac{n_k}{n}$$

for each k , where $n_k = |\{i: y^{(i)} = k\}|$ is the number of training targets with class k .

Thus, the optimal ϕ_k is just the proportion of data points with class k in the training set!

Learning the Parameters ψ_{jk}

Next we want to learn the parameter ψ_{jk} (i.e., probability that $x_j = 1$ given class k).

- For each k , consider all the inputs x for which $y = k$.
- We seek the probability ψ_{jk} of a word j being present in an x .

What is the optimal ψ_{jk} in this case?

Each ψ_{jk} is simply the proportion of documents in class k that contain the word j .

$$\psi_{jk} = \frac{n_{jk}}{n_k}.$$

where $|\{i : x_j^{(i)} = 1 \text{ and } y^{(i)} = k\}|$ is the number of $x^{(i)}$ with label k and a positive occurrence of word j .

Querying the Model

How do we ask the model for predictions? As discussed earlier, we can apply Bayes' rule:

$$\arg \max_y P_{\theta}(y|x) = \arg \max_y P_{\theta}(x|y)P(y).$$

Thus, we can estimate the probability of x and under each $P_{\theta}(x|y = k)P(y = k)$ and choose the class that explains the data best.

Classification Dataset: Twenty Newsgroups

To illustrate the text classification problem, we will use a popular dataset called **20-newsgroups**.

- It contains 20,000 documents collected approximately evenly from 20 different online newsgroups.
- Each newsgroup covers a different topic such as medicine, computer graphics, or religion.
- This dataset is widely used to benchmark text classification and other types of algorithms.

Classification Dataset: Twenty Newsgroups

Let's load this dataset.

```
#https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html
from sklearn.datasets import fetch_20newsgroups

# for this lecture, we will restrict our attention to just 4 different
categories = ['alt.atheism', 'soc.religion.christian', 'comp.graphics', 'sci.space']
twenty_train = fetch_20newsgroups(subset='train', categories=categories)
Markdown(twenty_train.DESCR[:1088]);
```

Example: Text Classification

Let's see how this approach can be used in practice on the text classification dataset.

- We will learn a good set of parameters for a Bernoulli Naive Bayes model
- We will compare the outputs to the true predictions.

Let's see an example of Naive Bayes on 20-newsgroups.

We start by computing these features using the `sklearn` library.

```
from sklearn.feature_extraction.text import CountVectorizer

# vectorize the training set
count_vect = CountVectorizer(binary=True, max_features=1000)
y_train = twenty_train.target
X_train = count_vect.fit_transform(twenty_train.data).toarray()
feature_names = count_vect.get_feature_names_out()
X_train.shape
```

(2257, 1000)

Example: Text Classification

Let's compute the maximum likelihood model parameters on our dataset. This is done in closed-form by just estimating statistics on the data, so we don't need to do gradient descent.

```
n = X_train.shape[0] # size of the dataset
d = X_train.shape[1] # number of features in our dataset
K = 4 # number of classes
```

```
# these are the shapes of the parameters
```

```
psis = np.zeros([K,d])
```

```
phis = np.zeros([K])
```

```
# we now compute the parameters
```

```
for k in range(K):
```

```
    X_k = X_train[y_train == k]
```

```
    psis[k] = np.mean(X_k, axis=0)
```

```
    phis[k] = X_k.shape[0] / float(n)
```

```
# print out the class proportions
```

```
print(phis)
```

Example: Text Classification

The learned parameters contain information about the **most frequent words**:

```
top_words = []
for category in range(4):
    top_words.append(feature_names[np.argsort(psis[category])[-200:]])

for category in range(4):
    words_from_other_categories = np.concatenate(top_words[:category])
    unique_words = [word for word in top_words[category] if word not in words_from_other_categories]
    print(f'\n# top 10 ~unique words occurring in {twenty_train.target_names[category]}')
    print(unique_words[-10:])

# top 10 ~unique words occurring in alt.atheism:
['someone', 'again', 'allan', 'political', 'schneider', 'atheism', 'caltech', 'cco', 'keith', 'atheists']

# top 10 ~unique words occurring in comp.graphics:
['files', 'version', 'file', 'mail', 'image', 'keywords', 'program', 'looking', 'please', 'graphics']

# top 10 ~unique words occurring in sci.med:
['soon', 'univ', 'pittsburgh', 'disease', 'geb', 'banks', 'medical', 'years', 'gordon', 'pitt']

# top 10 ~unique words occurring in soc.religion.christian:
['faith', 'athos', 'church', 'bible', 'christ', 'jesus', 'apr', 'christians', '1993', 'rutgers']
```

Example: Text Classification

We can compute **predictions** using Bayes' rule.

```
def nb_predictions(x, psis, phis):  
    """This returns class assignments and scores under the NB model.
```

```
We compute  $\arg\max_y p(y|x)$  as  $\arg\max_y p(x|y)p(y)$   
    """
```

```
# adjust shapes
```

```
n, d = x.shape
```

```
x = np.reshape(x, (1, n, d))
```

```
psis = np.reshape(psis, (K, 1, d))
```

```
psis = psis.clip(1e-14, 1-1e-14) # clip probabilities to avoid 1
```

```
# compute log-probabilities
```

```
logpy = np.log(phis).reshape([K,1])
```

```
logpxy = x * np.log(psis) + (1-x) * np.log(1-psis)
```

```
logpyx = logpxy.sum(axis=-2) + logpy
```

Example: Text Classification

```
return logpyx.argmax(axis=0).flatten(), logpyx.reshape([K,n])
```

```
idx_train, logpyx = nb_predictions(X_train, psis, phis)  
print(idx_train[:10])
```

```
[1 1 3 0 3 3 3 2 2 2]
```

We can measure the **accuracy**:

```
(idx_train==y_train).mean()
```

```
0.8692955250332299
```

Example: Text Classification

We can also **make up new queries** and **ask the model** to label them:

```
docs_new = [ 'OpenGL on the GPU is fast ' ]

X_new = count_vect.transform(docs_new).toarray()
predicted , logpyx_new = nb_predictions(X_new, psis , phis)

for doc, category in zip(docs_new, predicted):
    print('%r => %s' % (doc, twenty_train.target_names[category]))

    'OpenGL on the GPU is fast' => comp.graphics
```